

Nắm vững bản chất, vận dụng linh hoạt: bàn sâu về quy hoạch động

Lai Yukun

Nguồn gốc: Grasping the essence and applying flexibly: an in-depth discussion of dynamic programming

IOI China candidate team papers / 1999 / Lai Yukun.pdf

Từ khóa. Quy hoạch động; xây dựng ý tưởng; cài đặt.

Tóm Tắt

Bài viết này thảo luận nội dung cốt lõi và các đặc điểm cơ bản của tư tưởng quy hoạch động; khảo sát phạm vi áp dụng của nó, cũng như cách suy nghĩ chung khi xác lập không gian bài toán con và công thức truy hồi. Thông qua các ví dụ, bài viết chỉ ra một số cách xử lý trong quá trình xác lập bài toán con: tăng cường mệnh đề, điều chỉnh hợp lý các biến xác định trạng thái để giúp lập phương trình quy hoạch động, và dùng tiền xử lý để quá trình quy hoạch động dễ cài đặt hơn. Tiếp đó, bài viết phân tích vấn đề tràn bộ nhớ có thể xuất hiện khi cài đặt quy hoạch động và một số cách khắc phục. Kết luận của bài viết là: điểm then chốt của quy hoạch động vẫn nằm ở việc lập mô hình toán học hiệu quả cho từng bài toán khác nhau, rồi vận dụng linh hoạt trên cơ sở nắm chắc bản chất.

1 Dẫn nhập

Quy hoạch động là một tư tưởng thiết kế chương trình quan trọng và có giá trị ứng dụng rộng rãi. Với khá nhiều bài toán, dùng tư tưởng quy hoạch động để thiết kế thuật toán thường đạt hiệu quả thời gian cao; vì vậy, đối với những bài có thể giải bằng quy hoạch động, đây thường là lựa chọn sáng suốt.

Những bài toán có thể giải bằng quy hoạch động thường là bài toán tối ưu, và một phần cục bộ của lời giải tối ưu, hoặc của lời giải đặc thù, thường cũng là lời giải tối ưu của bài toán con tương ứng dưới những điều kiện phù hợp. Đồng thời, lời giải tối ưu của bài toán phải có liên hệ nhất định với lời giải tối ưu của các bài toán con để có thể thiết lập quan hệ truy hồi. Nếu quan hệ này khó thiết lập, tức là lời giải đặc thù của bài toán không chỉ phụ thuộc vào lời giải đặc thù của bài toán con mà còn phụ thuộc vào các lời giải tổng quát của bài toán con, thì một mặt ta khó ghi lại quá nhiều “lời giải tổng quát”, mặt khác hiệu suất truy hồi cũng rất thấp. Ngoài ra, để thể hiện hiệu quả thời gian cao của quy hoạch động, các bài toán con nên chồng lặp với nhau: nhiều bài toán khác nhau cùng dùng chung một số bài toán con. Nếu các bài toán con không chồng lặp, thường nên dùng phương pháp khác, như chia để trị.

Nói chung, quy hoạch động có thể được cài đặt khá hiệu quả bằng hai cách. Cách thứ nhất là ghi nhớ từ trên xuống: dùng đệ quy hoặc cách không đệ quy để quy việc tìm lời giải tối ưu của bài toán về việc tìm lời giải tối ưu của các bài toán con, rồi ghi lại các kết quả đã tính để chia sẻ. Cách thứ hai, cũng là cách chủ yếu, là truy hồi từ dưới lên. Vì chi phí của cách này nhỏ hơn cách thứ nhất nên nó được dùng phổ biến; các thảo luận dưới đây đều xét cách cài đặt này. Quy hoạch động có hiệu quả cao vì trong khi

liên tục giảm quy mô bài toán, nó ghi lại lời giải một cách hữu hiệu, tránh hiện tượng giải đi giải lại cùng một bài toán con. Do đó, nếu vận dụng đúng, hiệu suất thường cao hơn tìm kiếm rất nhiều.

Tư tưởng quy hoạch động cho ta một hướng suy nghĩ khi giải các bài toán tối ưu liên quan đến bài toán con chồng lặp: xét các bài toán con theo vòng lặp, giảm quy mô bài toán rồi cuối cùng giải được bài toán ban đầu. Phạm vi bài toán thích hợp với quy hoạch động rất rộng. Bản thân tư tưởng quy hoạch động là quan trọng, nhưng điều quan trọng hơn là phân tích cụ thể từng bài toán: xem bài toán có đủ điều kiện dùng quy hoạch động hay không, xác định không gian bài toán con và công thức truy hồi, rồi cài đặt những thuật toán này trên máy tính có bộ nhớ thông thường hữu hạn. Dưới đây, ta lần lượt bàn sâu hơn về hai phương diện: xây dựng ý tưởng và cài đặt.

2 Xây dựng ý tưởng giải bài bằng quy hoạch động

Khi đối diện một bài toán và cân nhắc dùng quy hoạch động, điểm rất quan trọng là phán đoán bài toán ấy có thể được giải hiệu quả bằng quy hoạch động hay không. Khi xây dựng thuật toán quy hoạch động, ta thường phải xét các bài toán con liên quan, tức không gian bài toán con, cách lập công thức truy hồi, và cuối cùng là cài đặt thuật toán. Thực ra các quá trình này thường đan xen với nhau. Không gian bài toán con và quan hệ truy hồi vốn liên hệ chặt chẽ: để lập được quan hệ truy hồi hiệu quả, đôi khi cần điều chỉnh không gian bài toán con; ngược lại, không gian bài toán con đã được xác định sơ bộ cũng có thể gợi ý cho ta cách lập công thức truy hồi. Việc cuối cùng có thể dùng một công thức truy hồi để nối bài toán với các bài toán con hay không cũng trở thành căn cứ chính để phán đoán bài toán có thể dùng quy hoạch động hay không. Vì thế, tách riêng từng phần để xét cô lập, hay cố chia quá trình suy nghĩ thành vài đoạn cứng nhắc, vừa khó vừa không cần thiết. Hơn nữa, quy hoạch động không có một mô hình toán học cố định; nó thay đổi theo từng bài toán, nên không thể quy nạp thành một phương pháp “vạn năng”. Dù vậy, với đa số bài toán, vẫn có một hướng suy nghĩ cơ bản.

Trước hết, cần phân tích sơ bộ xem bài toán có khả năng giải bằng quy hoạch động hay không. Nếu một bài toán khó xác định bài toán con, hoặc giữa lời giải đặc thù của bài toán và lời giải đặc thù của bài toán con hoàn toàn không có liên hệ, ta nên cân nhắc các phương pháp khác như tìm kiếm. Phán đoán sơ bộ là cần thiết, vì nó giúp tránh lãng phí thời gian. Thông thường, một bài toán có thể dùng quy hoạch động hiệu quả về cơ bản thỏa các đặc điểm sau:

1. Lời giải tối ưu của bài toán con chỉ liên quan đến điểm đầu và điểm cuối, hoặc các đại lượng đại diện tương ứng, chứ không phụ thuộc vào đường đi đã dùng để tới điểm đầu, điểm cuối ấy.
2. Có rất nhiều bài toán con chồng lặp; nếu không, ưu thế của quy hoạch động khó thể hiện.

Sau đây ta lấy bài “Nhận dạng ký tự” của IOI 1997 để phân tích cách hình thành suy nghĩ quy hoạch động trong tình huống thông thường.

Bài toán nhận dạng ký tự của IOI 1997. Đề bài như sau. Tập FONT.DAT mô tả dạng chữ của 27 ký hiệu: khoảng trắng và các chữ cái A đến Z. Ma trận điểm ảnh của mỗi ký hiệu được biểu diễn bằng 20 dòng, mỗi dòng gồm 20 ký tự 0 hoặc 1. Một tệp nhập khác mô tả ảnh điểm của một chuỗi ký tự gồm N dòng, nhưng các ký tự có thể bị “hỏng”: một số 0 biến thành 1, hoặc một số 1 biến thành 0. Mỗi dòng vẫn luôn có 20 ký tự 0 hoặc 1, nhưng số dòng ứng với mỗi ký tự có thể rơi vào một trong các trường hợp:

- vẫn là 20 dòng, tức không mất dòng nào và không dòng nào bị sao chép;
- là 19 dòng, tức có một dòng bị mất;
- là 21 dòng, tức có một dòng bị sao chép; hai dòng sao chép có thể bị hỏng khác nhau.

Yêu cầu là, dưới một giả thiết phân đoạn dòng nào đó, xuất ra kết quả nhận dạng, tức chuỗi ký tự ứng với phương án làm cho số lần đảo 0 và 1 là ít nhất.

Sau khi sơ bộ xác định bài toán này có thể giải bằng quy hoạch động, ta có thể dùng phương pháp, hoặc góc nhìn, toán học để mô tả bài toán. Với một bài toán tối ưu thông thường, vốn cũng là loại bài chính của quy hoạch động, luôn có một tiêu chuẩn tối ưu hóa. Vì quy hoạch động phải được thực hiện bằng truy hồi, ta cần phân tích các đại lượng cần thiết để xác định trạng thái. Chẳng hạn, trong bài nhận dạng ký tự, xét theo quy mô bài toán thì ta đang tìm một cách phân đoạn và đối ứng cho N dòng. Vì thế, xét vài dòng đầu là một đại lượng xác định trạng thái rất tự nhiên. Tiêu chuẩn tối ưu đã được đề bài cho: dưới một giả thiết nào đó, bao gồm cách phân đoạn và cách nhận dạng, số lần đảo 0 và 1 là ít nhất.

Nếu xem tiêu chuẩn đo lường này là một hàm, thì nó chính là một hàm tối ưu, hay hàm mục tiêu. Giá trị của hàm tối ưu phụ thuộc vào các biến độc lập, tức các đại lượng xác định trạng thái. Số biến độc lập phải tùy bài toán; ở đây chỉ có một biến là số dòng, mang nghĩa “xét vài dòng đầu”. Điểm then chốt là xác định trạng thái một cách hữu hiệu. Với trạng thái ấy, các đại lượng đã chọn phải đủ để xác định giá trị hàm tối ưu; về lý thuyết, khi các đại lượng ấy đã cố định, hàm tối ưu phải có một giá trị xác định. Nếu không, đại lượng đã chọn là chưa đủ, hoặc cần dùng đại lượng khác để mô tả. Ngay cả khi đã đủ xác định, nếu việc lập quan hệ truy hồi gặp khó khăn thì cũng phải điều chỉnh các biến xác định giá trị hàm tối ưu theo khó khăn ấy. Ngược lại, nếu đặt quá nhiều đại lượng để xác định hàm tối ưu, hiệu suất quy hoạch động sẽ giảm mạnh: hoặc phải giải nhiều bài toán con không cần thiết, hoặc biến các bài toán con chồng lặp thành những bài toán không chồng lặp dưới hệ biến ấy, làm hiệu suất giảm nghiêm trọng, thậm chí mất hẳn ưu thế của quy hoạch động. Trong ví dụ này, với L dòng đầu, hàm tối ưu rõ ràng có giá trị xác định.

Một tư tưởng quan trọng của truy hồi trong quy hoạch động là phân giải bài toán phức tạp thành các bài toán con. Vì thế, việc xác định không gian bài toán con và lập quan hệ truy hồi là rất quan trọng. Các biến xác định giá trị hàm tối ưu thường gợi ý cho không gian bài toán con. Thông thường, bằng cách suy ngược từ bước gần bài toán nhất, ta có thể thu được một bài toán con có quy mô nhỏ hơn; lặp lại cách xét ấy, ta thường cảm nhận được không gian bài toán con của bài toán. Trong quá trình đó, phân tích ngược cũng giúp rút ra quan hệ truy hồi tương đối dễ dàng. Cần nhấn mạnh rằng đây chỉ là tổng kết quá trình suy nghĩ khi giải một số bài; về nguyên tắc, các bài khác nhau vẫn cần được đối xử khác nhau.

Trong bài nhận dạng ký tự, khi xét giá trị hàm tối ưu cho n dòng, ta chú ý rằng cuối cùng chắc chắn phải phân đoạn và nhận dạng theo từng ký tự. Do đó, ký tự cuối cùng chỉ có thể gồm 19, 20, hoặc 21 dòng. Xét lần lượt ba cách phân đoạn này. Với đoạn vừa cắt ra ứng với một ký tự, trong mỗi phương án cắt đương nhiên phải chọn ký tự khớp nhất: nếu không dùng ký tự có số lần đảo ít nhất làm kết quả khớp mà vẫn dẫn tới tối ưu toàn cục, thì chỉ cần thay bước này bằng ký tự có số lần đảo ít nhất sẽ thu được kết quả tốt hơn “tối ưu” đã giả định, mâu thuẫn. Sau khi bỏ đi một ký tự, số dòng giảm xuống; những dòng còn lại rõ ràng cũng phải được khớp theo cách tối ưu, điều này có thể chứng minh phản chứng tương tự. Như vậy ta thu được một bài toán con giống bài toán ban đầu, cùng biến xác định và cùng tiêu chuẩn tối ưu, nhưng quy mô nhỏ hơn. Đồng thời, quan hệ truy hồi giữa bài toán con và bài toán ban đầu cũng được thiết lập:

$$\begin{aligned} f[i] = \min\{ & \text{Compare19}[i - 19 + 1] + f[i - 19], \\ & \text{Compare20}[i - 20 + 1] + f[i - 20], \\ & \text{Compare21}[i - 21 + 1] + f[i - 21]\}. \end{aligned}$$

Ở đây $f[i]$ biểu thị giá trị hàm tối ưu khi khớp i dòng đầu; $\text{Compare19}[i]$, $\text{Compare20}[i]$, và $\text{Compare21}[i]$ lần lượt biểu thị số ký tự 0 và 1 bị đảo ít nhất khi 19, 20, hoặc 21 dòng bắt đầu từ dòng i được khớp với ký tự gần nhất theo ba cách khớp tương ứng. Điều kiện đầu là $f[0] = 0$; với số dòng không thể khớp, chỉ cần dùng một số rất lớn đặc biệt để biểu diễn. Tất nhiên, trọng tâm của bài này không chỉ nằm ở suy nghĩ quy hoạch động cơ bản. Ví dụ này chỉ dùng để nói về một hướng suy nghĩ cơ bản cho nhiều bài quy hoạch động; bài còn có phần mô hình hóa toán học như dùng nhị phân để biểu diễn chuỗi 0, 1. Mã

nguồn xem Chương trình 1 trong phụ lục.

Đôi khi, dù các đại lượng xác định trạng thái thu được theo hướng trên đã đủ để làm hàm tối ưu có giá trị xác định, việc lập quan hệ truy hồi vẫn gặp khó khăn. Khi ấy, đưa thêm biến mới hoặc điều chỉnh biến đã có cũng là một cách khắc phục. Chẳng hạn, xét bài “Trò chơi xếp khối” của NOI 1997.

Bài toán xếp khối. Khối là một hình hộp chữ nhật. Biết độ dài ba cạnh a, b, c của N khối có đánh số, cần chọn một số khối trong N khối để xếp thành M cột ($1 \leq M \leq N \leq 100$), sao cho tổng chiều cao lớn nhất và thỏa:

- mọi khối trong cột thứ K có số hiệu lớn hơn mọi khối trong cột thứ $K + 1$;
- với hai khối kề nhau bất kỳ, mặt trên của khối dưới phải chứa được mặt dưới của khối trên, và số hiệu khối dưới phải nhỏ hơn số hiệu khối trên.

Vì đề yêu cầu cột có số thứ tự nhỏ hơn chứa các khối có số hiệu lớn hơn, điều này không thật tự nhiên. Không làm thay đổi kết quả, ta đổi thành cột có số thứ tự nhỏ hơn chứa các khối có số hiệu nhỏ hơn; điều này hiển nhiên không ảnh hưởng đến tổng chiều cao cuối cùng. Khi đó, mỗi cách xếp hợp lệ có thể xem là: chọn một số khối theo thứ tự tăng dần của số hiệu, đặt từng cột theo thứ tự tăng dần của số cột, và trong mỗi cột thì các khối lần lượt được đặt lên trên khối trước để tạo thành một phương án.

Dùng phương pháp phân tích chung ở trên, ta dễ thấy rằng có thể xét i khối đầu được xếp thành j cột đầu. Hai đại lượng i, j rõ ràng xác định được giá trị hàm tối ưu, nhưng quan hệ truy hồi lại không dễ lập trực tiếp. Phân tích thêm sẽ thấy vấn đề chính nằm ở việc khối thứ i rốt cuộc có thể đặt lên cột cuối cùng trong phương án tối ưu của bài toán con, tức trạng thái xác định bởi $(i - 1, j)$, hay không. Nếu xét thêm hai biến là ràng buộc tối thiểu về dài và rộng của mặt trên khối cuối cùng trong dãy, việc lập truy hồi không khó. Tuy nhiên, rõ ràng trong quá trình truy hồi có rất nhiều kết quả không được dùng, làm phình to không gian bài toán con một cách nhân tạo, gây rắc rối cho lưu trữ và làm hiệu suất thấp.

Thực ra, để lập truy hồi, ta chỉ cần biết mặt trên của cột cuối cùng trong lời giải bài toán con có chứa được khối hiện tại hay không. Những mặt ràng buộc có thể xuất hiện trong đề nhiều nhất chỉ có $3 \cdot 100 + 1 = 301$ loại, gồm cả trường hợp không ràng buộc. Như vậy, sau khi thêm một đại lượng là “yêu cầu mặt trên của cột cuối cùng chứa được loại mặt thứ k ”, quan hệ truy hồi chỉ cần chia thành ba trường hợp: khối hiện tại mở một cột mới, khối hiện tại đặt lên cột trước nếu có thể, hoặc không dùng khối hiện tại. Mã nguồn xem Chương trình 2.

Ngoài ra, có những bài toán vẫn khó lập truy hồi nếu chỉ điều chỉnh quan hệ truy hồi theo cách trên. Khi ấy, thêm đại lượng hoặc hàm khác để lập công thức truy hồi cũng là một hướng suy nghĩ, tương tự việc “tăng cường mệnh đề” trong chứng minh bằng quy nạp toán học. Một đặc trưng quan trọng khi dùng quy hoạch động là truy hồi; truy hồi là quá trình dùng kết quả cũ để thu được kết quả mới. Nếu về lý thuyết có thể chứng minh rằng một bài toán khó truy hồi trực tiếp có thể được giải cùng lúc với một quan hệ truy hồi mới đưa vào, thì nhìn qua có vẻ làm bài toán phức tạp hơn, nhưng thực tế ở mỗi bước truy hồi, ta vừa tăng thêm bài toán cần giải, vừa tăng thêm điều kiện, tức các giá trị đã giải trước đó, nên truy hồi lại dễ tiến hành hơn.

Ví dụ, xét bài “Đa giác” của IOI 1998. Bài toán như sau. Có một đa giác N cạnh, trên đỉnh đặt số nguyên, trên cạnh đặt dấu $+$ hoặc $*$. Cần tìm một thứ tự hợp nhất bằng phép toán, qua $N - 1$ phép tính, để kết quả cuối cùng là lớn nhất.

Nếu chỉ xét $\text{MAX}[I, L]$, là giá trị lớn nhất có thể thu được khi bắt đầu từ I và thực hiện L phép toán, thì khó truy hồi. Dựa vào kiến thức toán học, ta đưa thêm $\text{MIN}[I, L]$, là giá trị nhỏ nhất có thể thu được khi bắt đầu từ I và thực hiện L phép toán. Khi truy hồi, ta có thể dùng hiệu quả các giá trị với I, L nhỏ hơn để thu được kết quả lớn hơn, và thực chất giải đồng thời hai bài toán giá trị nhỏ nhất và lớn nhất.

Quan hệ truy hồi như sau. Xét I từ 1 đến N , L từ 1 đến $N - 1$. Xét t , vị trí phép toán cuối cùng, từ 0

đến $L - 1$. Nếu phép toán cuối cùng là $+$, thì:

$$\begin{aligned}\min(i, L) &= \min\{\min(i, t) + \min((i + t + 1 - 1) \bmod N + 1, L - t - 1)\}, \\ \max(i, L) &= \max\{\max(i, t) + \max((i + t + 1 - 1) \bmod N + 1, L - t - 1)\}.\end{aligned}$$

Nếu phép toán cuối cùng là $*$, thì:

$$\begin{aligned}\min(i, L) &= \min\{\min(i, t) \min((i + t + 1 - 1) \bmod N + 1, L - t - 1), \\ &\quad \min(i, t) \max((i + t + 1 - 1) \bmod N + 1, L - t - 1), \\ &\quad \max(i, t) \min((i + t + 1 - 1) \bmod N + 1, L - t - 1), \\ &\quad \max(i, t) \max((i + t + 1 - 1) \bmod N + 1, L - t - 1)\}, \\ \max(i, L) &= \max\{\min(i, t) \min((i + t + 1 - 1) \bmod N + 1, L - t - 1), \\ &\quad \min(i, t) \max((i + t + 1 - 1) \bmod N + 1, L - t - 1), \\ &\quad \max(i, t) \min((i + t + 1 - 1) \bmod N + 1, L - t - 1), \\ &\quad \max(i, t) \max((i + t + 1 - 1) \bmod N + 1, L - t - 1)\}.\end{aligned}$$

Mã nguồn xem Chương trình 3.

Ngoài ra, quy hoạch động được thực hiện bằng truy hồi, nên bài toán và bài toán con càng giống nhau, càng có quy luật thì càng dễ thao tác. Vì thế, với một số bài mà các giai đoạn và quy luật tự thân không rõ ràng, ta có thể dùng tiền xử lý để làm chúng đều đặn hơn, dễ cài đặt hơn. Ví dụ, xét bài “Khớp biểu thức chính quy” trong ACM Asia Regional 1997, khu vực Thượng Hải.

Bài toán khớp biểu thức chính quy. Biểu thức chính quy là biểu thức có ký tự đại diện. Các ký hiệu mở rộng được đề định nghĩa gồm:

- $.$: biểu thị ký tự bất kỳ;
- $[c1 - c2]$: biểu thị một ký tự bất kỳ nằm giữa $c1$ và $c2$;
- $[^c1 - c2]$: biểu thị một ký tự bất kỳ không nằm giữa $c1$ và $c2$;
- $*$: ký tự đứng trước nó có thể xuất hiện 0 hoặc nhiều lần;
- $+$: ký tự đứng trước nó có thể xuất hiện một hoặc nhiều lần;
- $\backslash$: ký tự đứng sau nó được xem như một ký tự thường.

Với một chuỗi nhập, cần tìm chuỗi con khớp với biểu thức chính quy và nằm trái nhất; nếu điều kiện trái nhất như nhau thì chọn chuỗi dài nhất.

Nếu không tiền xử lý, đôi khi một ký hiệu mở rộng có thể ứng với nhiều ký tự, đôi khi nhiều ký hiệu mở rộng lại chỉ ứng với một ký tự, gây nhiều rắc rối cho xử lý có hệ thống. Vì vậy cần chuẩn hóa biểu thức chính quy, sao cho hoặc một nút chỉ ứng với một ký tự, hoặc dùng một đánh dấu đặc biệt để chỉ nó có thể lặp nhiều lần. Định nghĩa kiểu bản ghi:

```
NodeType = Record
  StartChar: Char; {ky tu bat dau}
  EndChar: Char;   {ky tu ket thuc}
  Belong: Boolean;  {co thuoc khoang hay khong}
  Times: Boolean;   {False: phai xuat hien mot lan;
                    True: co the xuat hien nhieu lan hoac khong xuat hien}
End;
```

Sau khi tiền xử lý dữ liệu nhập, việc lập quan hệ truy hồi không còn khó. Dùng $\text{Pro}[i, j]$ biểu thị việc i nút biểu thức chính quy đầu có khớp một chuỗi con kết thúc tại ký tự thứ j hay không, với giá trị True/False. Với trường hợp True, đồng thời ghi rõ vị trí bắt đầu sớm nhất dưới điều kiện này. Nếu nút thứ i của biểu thức chính quy chỉ xuất hiện một lần, thì nếu nó không khớp ký tự thứ j , giá trị là False; nếu khớp, giá trị giống $\text{Pro}[i - 1, j - 1]$. Khởi đầu đặt $\text{Pro}[0, x] = \text{True}$. Nếu nút ấy có thể lặp nhiều lần, nó có thể được giải thích là 0 hoặc nhiều ký tự. Với điều kiện bản thân nút đó khớp 0 hoặc nhiều ký tự ở các vị trí tương ứng, ta xét lần lượt các khả năng này; chỉ cần trong đó có True thì $\text{Pro}[i, j]$ là True, đồng thời ghi lại vị trí bắt đầu sớm nhất trong các trường hợp đạt True. Truy hồi như vậy đến khi i đạt số nút. Mã nguồn xem Chương trình 4.

3 Các vấn đề khi cài đặt quy hoạch động

Sau khi đã có ý tưởng cơ bản để giải một bài bằng quy hoạch động, nhìn chung việc cài đặt thuật toán tương đối dễ suy nghĩ, nhưng đôi khi vẫn gặp một số vấn đề khiến thuật toán khó thực hiện. Về tổng thể, các thuật toán thiết kế theo tư tưởng quy hoạch động đều truy hồi theo công thức truy hồi đã rút ra. Đối với máy tính, nếu thiết kế hợp lý, loại truy hồi này thường có hiệu suất khá cao, nên khả năng quá thời gian không lớn. Ngược lại, quy hoạch động cần rất nhiều không gian để lưu các kết quả trung gian. Nhờ vậy, mọi bài toán chứa cùng một bài toán con có thể dùng chung lời giải của bài toán con ấy, thể hiện ưu thế của quy hoạch động; nhưng ưu thế này phải trả giá bằng không gian. Để truy cập hiệu quả các kết quả đã có, dữ liệu cũng không dễ nén khi lưu trữ, nên mâu thuẫn về không gian khá nổi bật.

Mặt khác, hiệu quả thời gian cao của quy hoạch động thường phải được thể hiện trên dữ liệu kiểm thử lớn, nhất là khi so với tìm kiếm. Vì vậy, với bài toán quy mô lớn, cách giải quyết vấn đề tràn bộ nhớ mà về cơ bản không ảnh hưởng đến tốc độ chạy là một vấn đề thường gặp khi dùng quy hoạch động.

Theo tôi, có thể thử suy nghĩ theo các hướng sau.

Một hướng là cố gắng dùng ít không gian nhất. Chẳng hạn, xét từ cấu trúc dữ liệu của nút, chỉ lưu những nội dung thật sự cần thiết; hoặc tính toán chặt chẽ phạm vi lưu trữ dữ liệu, như lưu theo bit, lưu nén, v.v. Tất nhiên, việc này phải phân tích theo từng bài. Ngoài ra, khi cài đặt quy hoạch động, một cách ta thường dùng là dùng một mảng có số phần tử bằng số nút để lưu quyết định ở mỗi bước. Điều này rất thuận tiện để suy ngược ra một phương án đạt lời giải tối ưu, và tốc độ xử lý cũng được cải thiện phần nào. Nhưng khi bộ nhớ căng thẳng, ta cần nắm mâu thuẫn chính của bài toán: bỏ mảng lưu quyết định ấy, đổi thành khi suy ngược từng cấp từ lời giải tối ưu, ta tính lại một lần nữa để chọn một trạng thái ở giai đoạn trước có thể đạt giá trị này, cho đến khi suy ra kết quả. Cách làm này tốn thêm một chút công viết chương trình so với cách trước, hiệu quả chạy cũng có thể giảm đôi chút, nhưng thường rất nhỏ, đổi lại tiết kiệm được nhiều không gian. Vì vậy tư tưởng này có ý nghĩa trong một số bài toán.

Tuy nhiên, đôi khi dù dùng cách ấy vẫn bị tràn không gian. Lúc đó cần phân tích xem các dữ liệu được giữ lại có thật sự phải đồng thời nằm trong bộ nhớ hay không. Trong rất nhiều bài, khi truy hồi quy hoạch động xử lý các phần phía sau, dữ liệu ở những phần khá xa phía trước thực tế không còn được dùng. Với loại bài này, ta có thể ghi đè dữ liệu lên những vị trí chắc chắn không còn dùng nữa, từ đó tái sử dụng không gian. Nếu dùng hữu hiệu cách này, ngay cả bài toán quy mô khá lớn cũng không đến mức tràn bộ nhớ. Để tìm ra phương án tối ưu, việc giữ lại quyết định ở từng bước vẫn cần thiết, và điều này cũng cần không gian.

Nói chung, cách này có thể được cài đặt theo hai hướng. Một là kết quả truy hồi chỉ dùng hai mảng Data1 và Data2. Mỗi lần lấy Data1 làm giai đoạn trước, suy ra mảng Data2, rồi sao chép Data2 đè lên Data1, lặp lại như vậy để thu được kết quả cuối cùng. Cách này có hạn chế: với bài toán mà truy hồi liên quan đến vài giai đoạn trước, nó khá phiền phức; hơn nữa mỗi cấp truy hồi lại phải sao chép nhiều nội dung, tác động càng lớn nếu bài liên quan đến nhiều giai đoạn trước.

Cách thứ hai là, với bài toán có thể liên quan đến N giai đoạn trước, lập mảng $\text{Data}[0..N]$, trong đó mỗi phần tử có cùng nội dung như $\text{Data1}/\text{Data2}$ ở cách trên. Khi đó, nếu không dùng cách tiết kiệm bộ nhớ thì truy cập chỉ số K ; còn khi dùng cách này, chỉ cần đổi thành truy cập $K \bmod (N + 1)$. So với cách không xử lý như vậy, mã cần sửa rất ít, tốc độ gần như không bị ảnh hưởng vì phép mod trên máy tính rất nhanh, và việc giữ lại các số lượng giai đoạn khác nhau cũng dễ thực hiện. Cách này áp dụng được cho khá nhiều bài, chẳng hạn bài “Miễn phí bánh nướng” của NOI 1998.

Bài toán miễn phí bánh nướng. Đề bài như sau. Có một sân khấu rộng W ô, với W lẻ và $1 \leq W \leq 99$, chiều cao H ô với $1 \leq H \leq 100$. Ở thời điểm 0, người chơi đứng ở chính giữa sân khấu. Mỗi đơn vị thời gian, người chơi có thể đi sang trái 2 ô, sang trái 1 ô, đứng yên, sang phải 1 ô, hoặc sang phải 2 ô. Mỗi chiếc bánh cho biết thời điểm và vị trí bắt đầu rơi, tốc độ rơi là số ô đi xuống trong 1 giây, và điểm số. Chỉ khi ở cuối một giây nào đó, bánh và người chơi ở cùng một ô thì bánh mới được xem là bắt được. Cần tìm một phương án di chuyển để tổng điểm lớn nhất. Chú ý rằng các bánh đã được sắp theo thời điểm bắt đầu rơi.

Nhìn vào bài toán, nghĩ đến quy hoạch động không quá khó. Nhưng đề quy định thời điểm bắt đầu rơi từ 0 đến 1000, và nếu xét đến thời điểm rơi cuối cùng thì có thể tới khoảng 1100; chiều rộng có thể tới 99. Nếu lấy cặp thời gian–vị trí làm yếu tố xác định trạng thái để truy hồi thì tốc độ không chậm, nhưng nếu dùng một mảng cho kết quả sau khi tiền xử lý dữ liệu ban đầu, tức mảng mô tả tại thời điểm nào, vị trí nào có thể nhận bao nhiêu điểm, một mảng cho truy hồi quy hoạch động, và một mảng ghi quyết định ở mỗi bước, thì do đề không chỉ rõ độ lớn điểm số, nếu hai mảng đầu dùng kiểu LongInt, mảng cuối dùng ShortInt, bộ nhớ cần khoảng $1100 \cdot 99 \cdot 9$ byte, tức khoảng 957 KB. Rõ ràng điều này không thể lưu nổi.

Nhưng chú ý rằng khi truy hồi, một khi giá trị lớn nhất ứng với một cặp (thời gian, vị trí) đã xác định, dữ liệu gốc ở vị trí ấy không còn dùng nữa. Vì vậy hai mảng có thể nhập làm một, chỉ cần $1100 \cdot 99 \cdot 5$ byte, khoảng 532 KB. Với quy mô của đề, như vậy vừa đủ để giải quyết. Tất nhiên, nếu suy nghĩ thêm, trong bài này truy hồi thực ra chỉ liên quan đến thời điểm trước đó, còn dữ liệu bánh chỉ dùng giá trị tại vị trí hiện tại. Vì thời điểm bắt đầu rơi đã được sắp xếp, khi đọc tới một bánh có thời điểm bắt đầu muộn hơn thời điểm đang xử lý, không cần đọc ngay. Để tránh đọc đĩa lặp lại và thiếu quy luật, đồng thời tránh chi phí bộ nhớ quá lớn, chỉ cần ghi lại tình hình trong khoảng 100 đơn vị thời gian sau thời điểm hiện tại. Dùng phương pháp tái sử dụng bộ nhớ vòng đã nói ở trên, chỉ cần $101 \cdot 99 \cdot 4 + 99 \cdot 2 \cdot 2 = 40392$ byte, chưa tới 40 KB. Với mỗi thời điểm, chỉ cần 99 số ShortInt để lưu quyết định; dù nâng quy mô bài toán lên 3000 hoặc 4000 đơn vị thời gian cũng có thể giải thuận lợi. Mã nguồn xem Chương trình 5.

Khi những phương pháp trên vẫn không giải quyết được vấn đề bộ nhớ, cũng có thể dùng cấp phát động để khiến phần lớn điểm kiểm thử được giải hiệu quả. Ngoài ra, dùng bộ nhớ động còn có một lợi ích: khi cần trao đổi trong quá trình dùng lại bộ nhớ, ta có thể chỉ hoán đổi con trỏ mà không sao chép dữ liệu. Cách này cũng rất hữu hiệu trong thi đấu.

4 Kết luận

Quy hoạch động là một tư tưởng thiết kế chương trình quan trọng. Vì nó không có hình thức thuật toán cố định nên có tính linh hoạt lớn, nhưng bản chất của nó lại có mức độ tương đồng rất cao. Vì thế, khi học và sử dụng tư tưởng này, điểm then chốt là vận dụng linh hoạt trên cơ sở hiểu và nắm chắc bản chất. Bài viết tuy đã nói đến một số phương pháp tư duy, nhưng đó chỉ là thảo luận về một số vấn đề khá phổ biến; điều quan trọng nhất vẫn là phân tích cụ thể từng bài toán cụ thể và lập mô hình toán học phù hợp.

Tài liệu tham khảo

1. Wu Wenhui, Wang Jiande, *Phân tích thuật toán thực dụng và thiết kế chương trình*, Nhà xuất bản Công nghiệp Điện tử, ISBN 7-5053-4402-1/TP.2036.
2. Wu Wenhui, Wang Jiande, *Hướng dẫn thi Olympic Tin học quốc tế và quốc gia cho thanh thiếu niên: thuật toán và thiết kế chương trình trong tổ hợp*, Nhà xuất bản Đại học Thanh Hoa, ISBN 7-302-02203-8/TP.1060.
3. Đề NOI 1997, NOI 1998, IOI 1997, IOI 1998, và đề ACM Asia Regional 1997.

A Chương trình

A.1 Chương trình 1: IOI 1997, Nhận dạng ký tự

Phương trình quy hoạch động cơ bản của bài “Nhận dạng ký tự” đã được giải thích trong phần chính. Ở đây bổ sung điểm then chốt để tăng tốc: khi so sánh có lệch dòng, cần nâng cao hiệu suất. Trường hợp thiếu một dòng hoặc thừa một dòng tuy có thể xuất hiện lệch, nhưng mỗi dòng chỉ có thể được so sánh với một trong hai dòng lân cận. Vì vậy, trước hết ghi lại các phép so sánh có thể xảy ra, rồi cộng dồn số đảo khi khớp không lệch và khi khớp lệch từ hai đầu đến một vị trí. Khi xét 20 trường hợp, chỉ cần một vòng lặp ngoài, không tính vòng lặp trong thủ tục so sánh một dòng, nên hiệu suất tăng đáng kể.

```
program Character_Recognition;
const
  cc: array [1..27] of char =
    (' ','A','B','C','D','E','F','G','H','I','J','K','L',
     'M','N','O','P','Q','R','S','T','U','V','W','X','Y','Z');
var
  f, f1, f2: text;
  font: array [1..540] of longint; {mỗi longint biểu diễn 20 bit}
  dd: array [1..1200] of longint;
  str: string;
  i, j, k: integer;
  t: word;
  ff: integer;
  bin: array [1..20] of longint;
  pro: array [0..1200] of word;
  sta: array [0..1200] of byte;
  bf: array [0..1200] of word;
  pf: array [1..21,0..1] of word;
  n: integer;

procedure getnum(var l: longint);
var
  i: integer;
begin
  l := 0;
  for i := 1 to 20 do
    if str[i] = '1' then inc(l, bin[i]);
```



```

end;

function compare0(a, b: longint): byte;
var
  k: byte;
  i: integer;
begin
  a := a xor b;
  k := 0;
  for i := 1 to 20 do
    if a and bin[i] <> 0 then inc(k);
  compare0 := k
end;

function compare20(k: integer; var ff: integer): word;
var
  i, j, t, s: word;
  best: word;
begin
  ff := 0;
  best := maxint;
  for t := 1 to 27 do
    begin
      j := 0;
      for i := 1 to 20 do
        begin
          s := compare0(font[(t-1)*20+i], dd[i+k-1]);
          inc(j, s);
        end;
      if j < best then begin best := j; ff := t end;
    end;
  compare20 := best;
end;

function compare19(k: integer; var ff: integer): word;
var
  i, j, t: word;
  best: word;
  l1, l2: array [0..20] of word;
  bb: integer;
begin
  ff := 0;
  best := maxint;
  for t := 1 to 27 do
    begin
      fillchar(l1, sizeof(l1), 0);
      fillchar(l2, sizeof(l2), 0);
      for i := 1 to 19 do

```

```

    for j := 0 to 1 do
        pf[i,j] := compare0(font[(t-1)*20+i+j], dd[k+i-1]);
    l1[1] := pf[1,0];
    for i := 2 to 19 do l1[i] := l1[i-1] + pf[i,0];
    l2[19] := pf[19,1];
    for i := 18 downto 1 do l2[i] := l2[i+1] + pf[i,1];
    bb := maxint;
    for i := 1 to 20 do
        if l1[i-1] + l2[i] < bb then bb := l1[i-1] + l2[i];
    if bb < best then begin best := bb; ff := t end;
end;
compare19 := best
end;

function compare21(k: integer; var ff: integer): word;
var
    i, j, t: word;
    best: word;
    l1, l2: array [0..22] of word;
    bb: integer;
begin
    ff := 0;
    best := maxint;
    for t := 1 to 27 do
        begin
            fillchar(l1, sizeof(l1), 0);
            fillchar(l2, sizeof(l2), 0);
            fillchar(pf, sizeof(pf), 0);
            for i := 1 to 21 do
                for j := 0 to 1 do
                    if not ((i = 21) and (j = 0)) and
                        not ((i = 1) and (j = 1)) then
                        pf[i,j] := compare0(font[(t-1)*20+i-j], dd[k+i-1]);
            l1[1] := pf[1,0];
            for i := 2 to 20 do l1[i] := l1[i-1] + pf[i,0];
            l2[22] := 0;
            for i := 21 downto 2 do l2[i] := l2[i+1] + pf[i,1];
            bb := maxint;
            for i := 1 to 20 do
                if l1[i-1] + l2[i+1] + pf[i,0] < bb then
                    bb := l1[i-1] + l2[i+1] + pf[i,0];
            if bb < best then begin best := bb; ff := t end;
        end;
        compare21 := best
    end;

begin
    assign(f, 'Font.Dat'); reset(f);

```

```

assign(f1, 'Image.dat'); reset(f1);
assign(f2, 'Image.out'); rewrite(f2);

readln(f, k);
bin[1] := 1;
for i := 2 to 20 do bin[i] := bin[i-1] * 2;

for i := 1 to k do
begin
    readln(f, str);
    getnum(font[i]);
end;

read(f1, n);
for i := 1 to n do
begin
    readln(f1, str);
    getnum(dd[i]);
end;

fillchar(pro, sizeof(pro), 255);
fillchar(sta, sizeof(sta), 0);
fillchar(bf, sizeof(bf), 255);
pro[0] := 0; sta[0] := 0; bf[0] := 0;

for i := 19 to n do
begin
    if (i >= 19) and (pro[i-19] <> 65535) then
    begin
        t := compare19(i-19+1, ff);
        if t + pro[i-19] < pro[i] then
        begin
            pro[i] := t + pro[i-19];
            sta[i] := ff;
            bf[i] := i - 19;
        end;
    end;
end;
if (i >= 20) and (pro[i-20] <> 65535) then
begin
    t := compare20(i-20+1, ff);
    if t + pro[i-20] < pro[i] then
    begin
        pro[i] := t + pro[i-20];
        sta[i] := ff;
        bf[i] := i - 20;
    end;
end;
if (i >= 21) and (pro[i-21] <> 65535) then

```

```

begin
  t := compare2l(i-2l+1, ff);
  if t + pro[i-2l] < pro[i] then
    begin
      pro[i] := t + pro[i-2l];
      sta[i] := ff;
      bf[i] := i - 2l;
    end;
  end;
end;

str := '';
i := n;
if pro[n] = 65535 then
begin
  writeln(f2, 'No answer. ');
  close(f1); close(f2); halt
end;
while i <> 0 do
begin
  str := cc[sta[i]] + str;
  i := bf[i];
end;
writeln(f2, str);
close(f1); close(f2)
end.

```

A.2 Chương trình 2: NOI 1997, Trò chơi xếp khối

Chương trình dùng truy hồi từng cấp để tránh tràn bộ nhớ.

```

program Toy_Bricks_Game;
type
  jmttype = array [0..100] of ^node;
  node = array [0..300] of longint;
var
  f1, f2: text;
  size: array [0..300,1..2] of word;
  num: integer;
  n, m: integer;
  i, j, k, t: integer;
  p, q, r: jmttype;
  aa: array [1..100,1..3] of word;
  ss: array [1..100,1..3] of word;

procedure sort(i: integer);
var
  j, k, t: integer;

```

```

begin
  for j := 1 to 2 do
    for k := j+1 to 3 do
      if aa[i,j] > aa[i,k] then
        begin
          t := aa[i,j]; aa[i,j] := aa[i,k]; aa[i,k] := t
        end;
      end;
    end;
  end;

function add(a1, a2: integer): integer;
var
  i: integer;
begin
  for i := 1 to num do
    if (a1 = size[i,1]) and (a2 = size[i,2]) then
      begin add := i; exit end;
    inc(num);
    size[num,1] := a1; size[num,2] := a2;
    add := num;
  end;

procedure preprocess;
var
  i: integer;
begin
  num := 0;
  for i := 1 to n do
    begin
      sort(i);
      ss[i,1] := add(aa[i,1], aa[i,2]);
      ss[i,2] := add(aa[i,1], aa[i,3]);
      ss[i,3] := add(aa[i,2], aa[i,3]);
    end;
  end;

procedure check(ii, nn, hh: integer);
begin
  if (p[ii-1]^0 >= 0) and
    (p[ii-1]^0 + hh >= q[ii]^j) then
    q[ii]^j := p[ii-1]^0 + hh;
  if (p[ii-1]^ss[ii,nn] >= 0) and
    (q[ii-1]^ss[ii,nn] + hh >= q[ii]^j) then
    q[ii]^j := q[ii-1]^ss[ii,nn] + hh;
end;

begin
  assign(f1, 'ToyBrick.in'); reset(f1);
  assign(f2, 'ToyBrick.out'); rewrite(f2);

```

```

readln(f1, n, m);
for i := 1 to n do readln(f1, aa[i,1], aa[i,2], aa[i,3]);

size[0,1] := 0; size[0,2] := 0;
preprocess;

for i := 0 to n do
begin
  new(p[i]); new(q[i]);
  fillchar(q[i]^, sizeof(q[i]^), 0);
end;

for t := 1 to m do
begin
  r := q; q := p; p := r;
  for i := 0 to n do fillchar(q[i]^, sizeof(q[i]^), 255);
  for i := t to n do
    for j := 0 to num do
      begin
        q[i]^[j] := q[i-1]^[j];
        if (aa[i,1] >= size[j,1]) and (aa[i,2] >= size[j,2]) then
          check(i, 1, aa[i,3]);
        if (aa[i,1] >= size[j,1]) and (aa[i,3] >= size[j,2]) then
          check(i, 2, aa[i,2]);
        if (aa[i,2] >= size[j,1]) and (aa[i,3] >= size[j,2]) then
          check(i, 3, aa[i,1]);
      end;
    end;
  writeln(f2, q[n]^[0]);
  close(f1); close(f2)
end.

```

A.3 Chương trình 3: IOI 1998, Đa giác

```

program Polygon;
var
  f1, f2: text;
  n: integer;
  data: array [1..50] of integer;
  sign: array [1..50] of char;
  i, j, k, l: integer;
  t, s, p: integer;
  ans: set of 1..50;
  best: integer;
  min, max: array [1..50,0..50] of integer;
  first: boolean;

procedure init;

```

```

var
  i: integer;
  ch: char;
begin
  readln(f1, n);
  for i := 1 to n do
  begin
    repeat read(f1, ch) until ch <> ' ';
    sign[i] := ch;
    read(f1, data[i]);
  end;
end;

begin
  assign(f1, 'Polygon.in'); reset(f1);
  assign(f2, 'Polygon.out'); rewrite(f2);
  init;
  best := -maxint - 1;
  ans := [];
  fillchar(max, sizeof(max), 0);
  fillchar(min, sizeof(min), 0);
  for j := 1 to n do
  begin
    max[j,0] := data[j];
    min[j,0] := data[j];
  end;

  for l := 1 to n-1 do
    for k := 1 to n do
      begin
        max[k,l] := -maxint - 1;
        min[k,l] := maxint;
        for t := 0 to l-1 do
          case sign[(k+t+1-1) mod n + 1] of
            't':
              begin
                s := max[k,t] + max[(k+t+1-1) mod n + 1, l-t-1];
                if s > max[k,l] then max[k,l] := s;
                s := min[k,t] + min[(k+t+1-1) mod n + 1, l-t-1];
                if s < min[k,l] then min[k,l] := s;
              end;
            'x':
              begin
                for p := 1 to 4 do
                  begin
                    case p of
                      1: s := max[k,t] * max[(k+t+1-1) mod n + 1, l-t-1];
                      2: s := max[k,t] * min[(k+t+1-1) mod n + 1, l-t-1];

```

```

        3: s := min[k,t] * max[(k+t+1-1) mod n + 1, l-t-1];
        4: s := min[k,t] * min[(k+t+1-1) mod n + 1, l-t-1];
    end;
    if s > max[k,l] then max[k,l] := s;
    if s < min[k,l] then min[k,l] := s;
end;
end;
end;
end;

for i := 1 to n do
    if max[i,n-1] > best then begin best := max[i,n-1]; ans := [i] end
    else if max[i,n-1] = best then include(ans, i);

writeln(f2, best);
first := true;
for i := 1 to n do
    if i in ans then
        begin
            if first then first := false else write(f2, ' ');
            write(f2, i);
        end;
    writeln(f2);
    close(f1); close(f2)
end.

```

A.4 Chương trình 4: ACM 1997 khu vực châu Á, Thượng Hải: khớp biểu thức chính quy

```

program Expression_Match;
type
    datatype = record
        st, ed: char;
        md: 0..1; {0: mot lan; 1: lap 0 hoac nhieu lan}
        mt: 0..1; {0: khop khi khong thuoc; 1: khop khi thuoc}
    end;
var
    f1, f2: text;
    s1, s2: string;
    len: integer;
    dd: array [1..80] of datatype;
    pro: array [0..80,0..80] of boolean;
    fr: array [0..80,0..80] of byte;
    i, j, k: integer;
    ok: boolean;
    bt, bj: integer;

procedure preprocess;
var

```



```

    i, j: integer;
begin
    i := 0; j := 0;
    while i < length(s1) do
    begin
        inc(i);
        case s1[i] of
            '.':
            begin
                inc(j);
                dd[j].md := 0; dd[j].mt := 1;
                dd[j].st := #0; dd[j].ed := #255;
            end;
            '*': dd[j].md := 1;
            '+':
            begin
                inc(j);
                dd[j] := dd[j-1];
                dd[j].md := 1;
            end;
            '\':
            begin
                inc(i); inc(j);
                dd[j].md := 0; dd[j].mt := 1;
                dd[j].st := s1[i]; dd[j].ed := s1[i];
            end;
            '[':
            begin
                inc(i); inc(j);
                dd[j].md := 0; dd[j].mt := 1;
                if s1[i] = '^' then begin inc(i); dd[j].mt := 0 end;
                if s1[i] = '\' then inc(i);
                dd[j].st := s1[i];
                inc(i, 2);
                if s1[i] = '\' then inc(i);
                dd[j].ed := s1[i];
                inc(i);
            end
            else
            begin
                inc(j);
                dd[j].st := s1[i]; dd[j].ed := s1[i];
                dd[j].mt := 1; dd[j].md := 0;
            end;
        end;
        len := j
    end;
end;

```

```

begin
  assign(f1, 'Match.in'); reset(f1);
  assign(f2, 'Match.out'); rewrite(f2);
  while true do
    begin
      readln(f1, s1);
      if s1 = 'end' then break;
      readln(f1, s2);
      preprocess;
      ok := false;
      fillchar(pro, sizeof(pro), false);
      fillchar(pro[0], sizeof(pro[0]), true);
      fillchar(fr, sizeof(fr), 0);
      bt := maxint; bj := 0;
      for i := 0 to length(s2) do fr[0,i] := i + 1;
      for i := 1 to len do
        for j := 1 to length(s2) do
          begin
            if dd[i].md = 0 then
              begin
                if (dd[i].mt = 0) xor (s2[j] in [dd[i].st..dd[i].ed]) then
                  begin
                    pro[i,j] := pro[i-1,j-1];
                    if pro[i,j] then fr[i,j] := fr[i-1,j-1];
                  end
                else pro[i,j] := false;
              end
            else
              begin
                for k := j downto 0 do
                  begin
                    if pro[i-1,k] then
                      begin
                        pro[i,j] := true;
                        if (fr[i,j] = 0) or (fr[i,j] > fr[i-1,k]) then
                          fr[i,j] := fr[i-1,k];
                      end;
                    if not ((dd[i].mt = 0) xor (s2[k] in [dd[i].st..dd[i].ed])) then
                      break;
                    end;
                  end;
                end;
              end;
            if (i = len) and pro[i,j] and (fr[i,j] <= bt) then
              begin
                ok := true;
                bt := fr[i,j];
                bj := j - bt + 1;
              end;
          end;
        end;
      end;
    end;
  end;

```

```

    end;

    if ok then
        if bj <> 0 then
            writeln(f2, copy(s2,1,bt-1), '(', copy(s2,bt,bj), ')',
                    copy(s2,bt+bj,length(s2)-bt-bj+1))
            else writeln(f2, '()', s2)
            else writeln(f2, s2);
        end;
        close(f1); close(f2)
    end.

```

A.5 Chương trình 5: NOI 1998, Miễn phí bánh nướng

Phương trình quy hoạch động là

$$\text{ans}[t,j] = \max_{k \in \{-2,-1,0,1,2\}} \text{ans}[t-1, j+k],$$

trong đó vị trí tương ứng phải đạt được.

```

program Pizza_For_Free;
type
    link = ^linetype;
    linetype = array [1..99] of shortint;
var
    f1, f2: text;
    w, h: integer;
    dd: array [0..100] of array [1..99] of longint;
    pro: array [0..3100] of link;
    dt: array [0..1,1..99] of longint;
    i, j, k, t: integer;
    bt, bj: integer;
    best: longint;
    maxt: integer;
    ans: array [1..3100] of shortint;
    _t, _j, _v: integer;
    _fz: longint;
    _saved: boolean;

procedure try_reading;
var
    t0, j, v: integer;
    fz: longint;
begin
    fillchar(dd[(t+100) mod 101], sizeof(dd[(t+100) mod 101]), 0);
    if _saved then
        begin
            if _t > t then exit;
            _saved := false;

```

```

    if (h-1) mod _v = 0 then
    begin
        inc(dd[(_t+(h-1) div _v) mod 101,_j], _fz);
        if (_t+(h-1) div _v) > maxt then maxt := _t+(h-1) div _v;
    end;
end;
if eof(f1) then exit;

while true do
begin
    if eof(f1) then exit;
    readln(f1, t0, j, v, fz);
    if t0 > t then
    begin
        _v := v; _t := t0; _j := j; _fz := fz;
        _saved := true;
        exit
    end;
    if (h-1) mod v <> 0 then continue;
    inc(dd[(t0+(h-1) div v) mod 101,j], fz);
    if t0+(h-1) div v > maxt then maxt := t0+(h-1) div v;
end;
end;

begin
    assign(f1, 'INPUT.TXT'); reset(f1);
    assign(f2, 'OUTPUT.TXT'); rewrite(f2);
    readln(f1, w, h);

    for i := 0 to 3100 do
    begin
        new(pro[i]);
        fillchar(pro[i]^, sizeof(pro[i]^), 0);
    end;
    for i := 0 to 1 do
        for j := 1 to 99 do
            dt[i,j] := -maxlongint - 1;

dt[0,(1+w) div 2] := 0;
fillchar(dd, sizeof(dd), 0);
t := 0; maxt := 0; _saved := false;
try_reading;
best := dd[0,(w+1) div 2];
bt := 0; bj := (w+1) div 2;

while true do
begin
    inc(t);

```

```

try_reading;
if eof(f1) and (t > maxt) and not _saved then break;
for j := 1 to w do
begin
  dt[t mod 2,j] := -maxlongint - 1;
  for k := -2 to 2 do
    if (j+k > 0) and (j+k <= w) and
      (dt[1-t mod 2,j+k] > -maxlongint - 1) and
      (dt[1-t mod 2,j+k] + dd[t mod 101,j] > dt[t mod 2,j]) then
    begin
      dt[t mod 2,j] := dt[1-t mod 2,j+k] + dd[t mod 101,j];
      pro[t]^j := k;
    end;
  if dt[t mod 2,j] > best then
  begin
    best := dt[t mod 2,j];
    bt := t; bj := j;
  end;
end;
end;

writeln(f2, best);
j := bj;
for t := bt downto 1 do
begin
  ans[t] := -pro[t]^j;
  inc(j, pro[t]^j);
end;
for t := 1 to bt do writeln(f2, ans[t]);
close(f1); close(f2)
end.

```

B Nội dung chi tiết của các bài toán được dẫn

B.1 Nhận dạng ký tự (IOI 1997)

Bài toán yêu cầu viết một chương trình thực hiện nhận dạng ký tự.

Mỗi ảnh ký tự lý tưởng có 20 dòng, mỗi dòng 20 chữ số; mỗi chữ số là 0 hoặc 1. Tập FONT.DAT chứa biểu diễn của 27 ảnh ký tự lý tưởng theo thứ tự: khoảng trắng, rồi a đến z.

Tập IMAGE.DAT chứa một hoặc nhiều ảnh ký tự có thể bị hỏng. Một ảnh ký tự có thể hỏng theo các cách sau:

- nhiều nhất một dòng bị sao chép, và dòng sao chép đứng ngay sau dòng gốc;
- nhiều nhất một dòng bị thiếu;
- một số ký tự 0 bị đổi thành 1;
- một số ký tự 1 bị đổi thành 0.

Không có ảnh ký tự nào vừa bị sao chép một dòng vừa bị thiếu một dòng. Trong dữ liệu đánh giá, không quá 30% số 0 và 1 của bất kỳ ảnh ký tự nào bị đổi. Khi một dòng bị sao chép, một hoặc cả hai dòng kết quả đều có thể bị hỏng, và hỏng theo các cách khác nhau.

Nhiệm vụ là nhận dạng chuỗi gồm một hoặc nhiều ký tự trong ảnh cho bởi IMAGE.DAT, dùng thông chữ cho bởi FONT.DAT. Khi nhận dạng một ảnh ký tự, hãy chọn ảnh ký tự trong thông chữ cần số lần đổi tổng cộng giữa 0 và 1 ít nhất để trở thành ảnh đã cho, dưới giả thiết thuận lợi nhất về dòng bị sao chép hoặc bị thiếu. Trong trường hợp dòng bị sao chép, chỉ tính lỗi trên dòng ít hỏng hơn. Mọi ký tự trong ảnh mẫu và ảnh đánh giá đều có thể được một chương trình tốt nhận dạng từng ký tự một. Mỗi bộ dữ liệu đánh giá có nghiệm tốt nhất duy nhất.

Một lời giải đúng phải dùng chính xác toàn bộ dữ liệu trong tệp IMAGE.DAT.

Dữ liệu vào. Cả hai tệp nhập đều bắt đầu bằng một số nguyên N , với $19 \leq N \leq 1200$, chỉ số dòng tiếp theo:

```
N
(digit1)(digit2)(digit3)...(digit20)
(digit1)(digit2)(digit3)...(digit20)
...
```

Mỗi dòng dữ liệu rộng 20 chữ số. Không có dấu cách giữa các số 0 và 1. Tệp FONT.DAT mô tả thông chữ, luôn chứa 541 dòng, và có thể khác nhau giữa các bộ kiểm thử.

Kết quả. Chương trình phải tạo tệp IMAGE.OUT, chứa một chuỗi duy nhất là các ký tự đã nhận dạng, trên một dòng văn bản ASCII. Kết quả không chứa ký tự phân cách. Nếu chương trình không nhận dạng được một ký tự nào đó, phải in ? ở vị trí tương ứng. Định dạng kết quả nói trên thay thế yêu cầu chuẩn trong quy chế về việc dùng dấu cách phân tách trong kết quả.

Điểm số được tính theo tỉ lệ ký tự nhận dạng đúng. Trong mẫu của đề, tệp IMAGE.DAT mô tả một ký tự A bị hỏng và kết quả mẫu là:

```
IMAGE.OUT
A
```

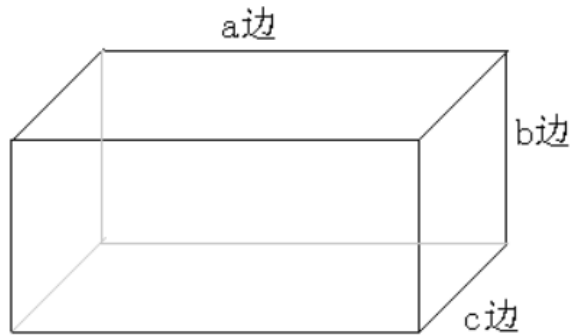
B.2 Trò chơi xếp khối (NOI 1997)

SERCOI gần đây thiết kế một trò chơi xếp khối. Mỗi người chơi có N khối hộp chữ nhật được đánh số lần lượt 1, 2, 3, 4, Ba cạnh khác nhau của mỗi khối lần lượt gọi là cạnh a , cạnh b , và cạnh c .

Luật chơi như sau:

1. Chọn một số khối trong N khối và chia chúng thành M cột ($1 \leq M \leq N$), gọi là cột 1, cột 2, cột 3, Mỗi cột có ít nhất một khối, và mọi khối trong cột thứ K phải có số hiệu lớn hơn mọi khối trong cột thứ $K + 1$.
2. Với mỗi cột, người chơi xếp các khối thẳng đứng thành một trụ, thỏa hai điều kiện: trừ khối trên cùng, mặt trên của mỗi khối tiếp xúc với đúng một mặt dưới của khối khác; mặt trên của khối dưới phải chứa được mặt dưới của khối trên, tức độ dài hai cặp cạnh của mặt trên khối dưới lần lượt lớn hơn hoặc bằng hai cặp cạnh của mặt dưới khối trên. Ngoài ra, với hai khối tiếp xúc trên dưới bất kỳ, số hiệu khối dưới phải nhỏ hơn số hiệu khối trên.

Cuối cùng, thắng thua được quyết định bởi tổng chiều cao của M trụ đã xếp. Hãy viết chương trình tìm một phương án xếp để tổng chiều cao của M trụ là lớn nhất.



Hình 1: Ba cạnh a, b, c của một khối hộp chữ nhật trong bài xếp khối.

Dữ liệu vào và kết quả. Tập nhập là INPUT.TXT. Dòng đầu có hai số nguyên dương N và M , ($1 \leq M \leq N \leq 100$), lần lượt là số khối và số trụ cần xếp. Hai số cách nhau bởi một dấu cách. Tiếp theo là N dòng theo thứ tự số hiệu khối từ 1 đến N ; mỗi dòng có ba số nguyên từ 1 đến 1000, là độ dài các cạnh a, b, c của khối ấy.

Tập ra là OUTPUT.TXT, chỉ có một dòng là một số nguyên, biểu thị tổng chiều cao của M trụ.

Ví dụ nhập:

INPUT.TXT

```
4 2
10 5 5
8 7 7
2 2 2
6 6 6
```

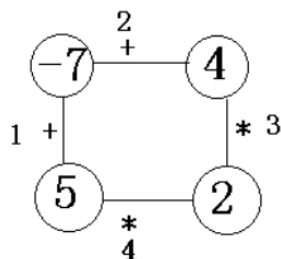
Ví dụ kết quả:

OUTPUT.TXT

24

B.3 Đa giác (IOI 1998)

Trò chơi đa giác là trò chơi một người. Ban đầu cho một đa giác có N đỉnh; mỗi đỉnh được gán một giá trị nguyên, mỗi cạnh được gán một ký hiệu $+$ hoặc $*$. Tất cả các cạnh được đánh số từ 1 đến N .



Hình 2: Ví dụ đa giác với $N = 4$, các đỉnh $-7, 4, 2, 5$ và các cạnh $1, \dots, 4$.

Nước đi đầu tiên cho phép xóa một cạnh. Mỗi nước đi tiếp theo gồm:

1. chọn một cạnh E , cùng hai đỉnh V_1 và V_2 nối bởi E ;

2. dùng một đỉnh mới thay thế cạnh E và hai đỉnh V_1, V_2 . Đỉnh mới được gán giá trị bằng kết quả áp dụng phép toán ghi trên E cho V_1 và V_2 .

Khi mọi cạnh đều bị xóa, chỉ còn một đỉnh và trò chơi kết thúc. Điểm của trò chơi là giá trị của đỉnh còn lại. Phần ví dụ chơi trong bản gốc chỉ ghi là không in tại đây.

Nhiệm vụ là viết chương trình, với một đa giác bất kỳ, tính điểm cao nhất có thể đạt được và liệt kê mọi cạnh có thể là cạnh bị xóa ở nước đầu tiên để dẫn đến điểm cao nhất ấy.

Tập `POLYGON.IN` gồm hai dòng. Dòng đầu chứa N . Dòng thứ hai chứa ký hiệu được gán cho từng cạnh $1, \dots, N$, cùng các giá trị đỉnh xen giữa hai cạnh. Giá trị nguyên đầu tiên ứng với đỉnh nối đồng thời cạnh 1 và cạnh 2; giá trị thứ hai ứng với đỉnh nối cạnh 2 và cạnh 3; cứ tiếp tục như vậy, giá trị cuối cùng ứng với đỉnh nối cạnh N và cạnh 1. Ký hiệu và giá trị cách nhau bằng một dấu cách. Ký hiệu cạnh có hai loại: chữ `t` ứng với $+$, và chữ `x` ứng với $*$.

Ví dụ nhập:

```
4
t -7 t 4 x 2 x 5
```

Tập này ứng với đa giác trong hình của đề; ký tự đầu tiên ở dòng hai là ký hiệu của cạnh 1.

Trong tập `POLYGON.OUT`, dòng đầu in điểm cao nhất có thể đạt được. Dòng thứ hai liệt kê theo thứ tự tăng dần mọi cạnh mà nếu xóa ở nước đầu tiên có thể dẫn đến điểm cao nhất, các số cách nhau bởi một dấu cách.

Ví dụ kết quả:

```
33
1 2
```

B.4 Khớp biểu thức chính quy (ACM 1997 khu vực châu Á, Thượng Hải)

Biểu thức chính quy là một chuỗi chứa các ký tự thường và một số ký tự meta. Các ký tự meta gồm:

- `.`: ký tự bất kỳ;
- `[c1-c2]`: ký tự bất kỳ nằm giữa $c1$ và $c2$;
- `[^c1-c2]`: ký tự bất kỳ không nằm giữa $c1$ và $c2$;
- `*`: ký tự trước nó có thể xuất hiện số lần bất kỳ, kể cả không xuất hiện;
- `+`: ký tự trước nó có thể xuất hiện số lần bất kỳ nhưng ít nhất một lần;
- `\`: mọi ký tự theo sau được xem là ký tự thường.

Cần viết chương trình tìm chuỗi con nằm trái nhất của một chuỗi cho trước sao cho chuỗi con ấy khớp với biểu thức chính quy đã cho. Nếu có nhiều chuỗi con cùng khớp và có cùng vị trí bắt đầu bên trái, chọn chuỗi dài nhất.

Dữ liệu vào. Mỗi hai dòng của tệp nhập là một bài toán khớp mẫu. Dòng đầu là biểu thức chính quy, dòng sau là chuỗi cần khớp. Mỗi dòng dài không quá 80 ký tự. Một dòng chỉ chứa end kết thúc dữ liệu vào.

Kết quả. Với mỗi bài toán, in một dòng. Dòng này chứa chuỗi cần khớp, nhưng chuỗi con trái nhất khớp với biểu thức chính quy phải được đặt trong ngoặc tròn. Nếu không có chuỗi con nào khớp, in nguyên chuỗi nhập.

Ví dụ nhập:

```
.*
asdf
f.*d.
sefdfsde
[0-9]+
asd345dsf
[^\*-\\*]
**asdf**fasd
b[a-z]*r[s-u]*
abcdefghijklmnopqrstuvwxy
z
[T-F]
dfkgj f
end
```

Ví dụ kết quả:

```
(asdf)
se(fdfsde)
asd(345)dsf
**(a)sdf**fasd
a(abcdefghijklmnopqrstu)vwxyz
dfkgj f
```

B.5 Miễn phí bánh nướng (NOI 1998)

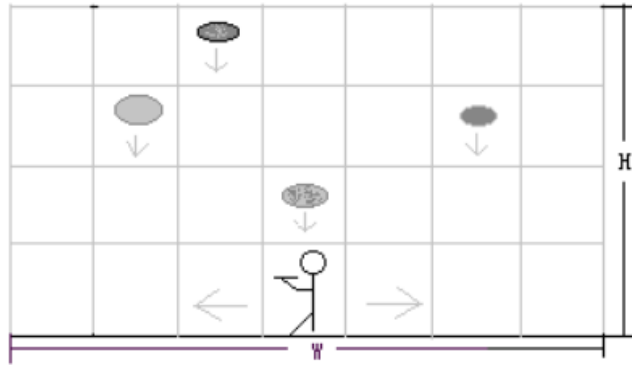
SERKOI vừa giới thiệu một trò chơi có tên “Miễn phí bánh nướng”.

Trò chơi diễn ra trên một sân khấu. Sân khấu rộng W ô, màn trời cao H ô, và người chơi chiếm một ô. Ban đầu người chơi đứng ở chính giữa sân khấu, tay cầm một chiếc khay. Sau khi trò chơi bắt đầu, bánh liên tục xuất hiện từ các ô ở mép trên màn trời và rơi thẳng xuống. Người chơi di chuyển trái phải để hứng bánh. Mỗi giây người chơi có thể sang trái hoặc sang phải một ô hoặc hai ô, cũng có thể đứng yên.

Có nhiều loại bánh. Người chơi đã chấm điểm từng loại bánh theo khẩu vị của mình. Dưới sự điều khiển từ xa của máy 8-308, tốc độ rơi của các loại bánh cũng khác nhau, tính bằng ô mỗi giây. Nếu ở cuối một giây nào đó, bánh vừa đến đúng ô người chơi đang đứng, người chơi thu được bánh ấy.

Hãy viết chương trình giúp người chơi thu bánh sao cho tổng điểm của các bánh thu được là lớn nhất.

Dữ liệu vào. Dòng đầu của tệp nhập gồm hai số nguyên dương cách nhau bởi dấu cách: chiều rộng W của sân khấu, là số lẻ từ 1 đến 99, và chiều cao H , từ 1 đến 100. Các dòng tiếp theo, theo thứ tự thời điểm bắt đầu rơi của bánh, cho thông tin về từng chiếc bánh. Mỗi dòng gồm bốn số nguyên dương: thời điểm bắt đầu rơi, từ 0 đến 1000 giây; vị trí ngang; tốc độ rơi, từ 1 đến 100; và điểm số. Thời điểm trò chơi bắt đầu là 0. Các ô theo chiều ngang được đánh số từ trái sang phải, bắt đầu từ 1. Các mục trên cùng một dòng được cách nhau bởi một hoặc nhiều dấu cách.



Hình 3: Minh họa sân khấu khi chiều cao màn trời là 4: bánh rơi từ phía trên, người chơi di chuyển ngang để hứng.

Kết quả. Dòng đầu của tệp ra in một số nguyên dương là tổng điểm lớn nhất chương trình có thể thu được. Mỗi dòng tiếp theo, theo thứ tự thời gian, in quyết định của người chơi trong từng giây. In 0 nghĩa là đứng yên; 1 hoặc 2 nghĩa là sang phải một hoặc hai ô; -1 hoặc -2 nghĩa là sang trái một hoặc hai ô. Kết quả kéo dài đến khi người chơi thu xong chiếc bánh cuối cùng mà phương án chọn sẽ thu.

Ví dụ nhập:

```
3 3
0 1 2 5
0 2 1 3
1 2 1 3
1 3 1 4
```

Ví dụ kết quả:

```
12
-1
1
1
```